```
template <int count>
class StringHashFamily
{
  public:
    StringHashFamily( ) : MULTIPLIERS( count )
    {
        generateNewFunctions( );
    }

    int getNumberOfFunctions( ) const
    {
        return count;
    }

    void generateNewFunctions( )
    {
        for( auto & mult : MULTIPLIERS )
            mult = r.nextInt( );
    }

    size_t hash( const string & x, int which ) const
    {
        const int multiplier = MULTIPLIERS[ which ];
        size_t hashVal = 0;

        for( auto ch : x )
            hashVal = multiplier * hashVal + ch;

        return hashVal;
    }

  private:
    vector<int> MULTIPLIERS;
    UniformRandom r;
};
```

Figure 5.45 Casual string hashing for cuckoo hashing; these hash functions do not provably satisfy the requirements needed for cuckoo hashing but offer decent performance if the table is not highly loaded and the alternate insertion routine in Figure 5.43 is used.

Finally, Figure 5.45 shows the `StringHashFamily` class that provides a set of simple hash functions for strings. These hash functions replace the constant 37 in Figure 5.4 with randomly chosen numbers (not necessarily prime).

The benefits of cuckoo hashing include the worst-case constant lookup and deletion times, the avoidance of lazy deletion and extra data, and the potential for parallelism.

However, cuckoo hashing is extremely sensitive to the choice of hash functions; the inventors of the cuckoo hash table reported that many of the standard hash functions that they attempted performed poorly in tests. Furthermore, although the insertion time is expected to be constant time as long as the load factor is below $\frac{1}{2}$, the bound that has been shown for the expected insertion cost for classic cuckoo hashing with two separate tables (both with load factor λ) is roughly $1/(1-(4\lambda^2)^{1/3})$, which deteriorates rapidly as the load factor gets close to $\frac{1}{2}$ (the formula itself makes no sense when λ equals or exceeds $\frac{1}{2}$). Using lower load factors or more than two hash functions seems like a reasonable alternative.

5.7.3 Hopscotch Hashing

Hopscotch hashing is a new algorithm that tries to improve on the classic linear probing algorithm. Recall that in linear probing, cells are tried in sequential order, starting from the hash location. Because of primary and secondary clustering, this sequence can be long on average as the table gets loaded, and thus many improvements such as quadratic probing, double hashing, and so forth, have been proposed to reduce the number of collisions. However, on some modern architectures, the locality produced by probing adjacent cells is a more significant factor than the extra probes, and linear probing can still be practical or even a best choice.

The idea of hopscotch hashing is to bound the maximal length of the probe sequence by a predetermined constant that is optimized to the underlying computer's architecture. Doing so would give constant-time lookups in the worst case, and like cuckoo hashing, the lookup could be parallelized to simultaneously check the bounded set of possible locations.

If an insertion would place a new item too far from its hash location, then we efficiently go backward toward the hash location, evicting potential items. If we are careful, the evictions can be done quickly and guarantee that those evicted are not placed too far from their hash locations. The algorithm is deterministic in that given a hash function, either the items can be evicted or they can't. The latter case implies that the table is likely too crowded, and a rehash is in order; but this would happen only at extremely high load factors, exceeding 0.9. For a table with a load factor of $\frac{1}{2}$, the failure probability is almost zero (Exercise 5.23).

Let *MAX_DIST* be the chosen bound on the maximum probe sequence. This means that item x must be found somewhere in the *MAX_DIST* positions listed in $hash(x)$, $hash(x)+1, \ldots, hash(x)+(MAX_DIST-1)$. In order to efficiently process evictions, we maintain information that tells for each position x, whether the item in the alternate position is occupied by an element that hashes to position x.

As an example, Figure 5.46 shows a fairly crowded hopscotch hash table, using $MAX_DIST = 4$. The bit array for position 6 shows that only position 6 has an item (C) with hash value 6: Only the first bit of Hop[6] is set. Hop[7] has the first two bits set, indicating that positions 7 and 8 (A and D) are occupied with items whose hash value is 7. And Hop[8] has only the third bit set, indicating that the item in position 10 (E) has hash value 8. If *MAX_DIST* is no more than 32, the Hop array is essentially an array of 32-bit integers, so the additional space requirement is not substantial. If Hop[*pos*] contains all 1s for some *pos*, then an attempt to insert an item whose hash value is *pos* will clearly

	Item	Hop
...		
6	C	1000
7	A	1100
8	D	0010
9	B	1000
10	E	0000
11	G	1000
12	F	1000
13		0000
14		0000
...		

A: 7
B: 9
C: 6
D: 7
E: 8
F: 12
G: 11

Figure 5.46 Hopscotch hashing table. The hops tell which of the positions in the block are occupied with cells containing this hash value. Thus Hop[8] = 0010 indicates that only position 10 currently contains items whose hash value is 8, while positions 8, 9, and 11 do not.

fail, since there would now be *MAX_DIST* + 1 items trying to reside within *MAX_DIST* positions of *pos*—an impossibility.

Continuing the example, suppose we now insert item *H* with hash value 9. Our normal linear probing would try to place it in position 13, but that is too far from the hash value of 9. So instead, we look to evict an item and relocate it to position 13. The only candidates to go into position 13 would be items with hash value of 10, 11, 12, or 13. If we examine Hop[10], we see that there are no candidates with hash value 10. But Hop[11] produces a candidate, *G*, with value 11 that can be placed into position 13. Since position 11 is now close enough to the hash value of *H*, we can now insert *H*. These steps, along with the changes to the Hop information, are shown in Figure 5.47.

Finally, we will attempt to insert *I* whose hash value is 6. Linear probing suggests position 14, but of course that is too far away. Thus we look in Hop[11], and it tells us that *G* can move down, freeing up position 13. Now that 13 is vacant, we can look in Hop[10] to find another element to evict. But Hop[10] has all zeros in the first three positions, so there are no items with hash value 10 that can be moved. So we examine Hop[11]. There we find all zeros in the first two positions.

So we try Hop[12], where we need the first position to be 1, which it is. Thus *F* can move down. These two steps are shown in Figure 5.48. Notice that if this were not the case—for instance if hash(*F*) were 9 instead of 12—we would be stuck and have to rehash. However, that is not a problem with our algorithm; instead, there would simply be no way to place all of *C*, *I*, *A*, *D*, *E*, *B*, *H*, and *F* (if *F*'s hash value were 9); these items would all have hash values between 6 and 9, and would thus need to be placed in the seven spots between 6 and 12. But that would be eight items in seven spots—an impossibility. However, since this is not the case for our example, and we have evicted an item from position 12, we can now continue. Figure 5.49 shows the remaining eviction from position 9 and subsequent placement of *I*.

	Item	Hop
...		
6	C	1000
7	A	1100
8	D	0010
9	B	1000
10	E	0000
11	G	1000
12	F	1000
13		0000
14		0000
...		

→

	Item	Hop
...		
6	C	1000
7	A	1100
8	D	0010
9	B	1000
10	E	0000
11		**0010**
12	F	1000
13	**G**	0000
14		0000
...		

→

	Item	Hop
...		
6	C	1000
7	A	1100
8	D	0010
9	B	**1010**
10	E	0000
11	**H**	0010
12	F	1000
13	G	0000
14		0000
...		

A: 7
B: 9
C: 6
D: 7
E: 8
F: 12
G: 11
H: 9

Figure 5.47 Hopscotch hashing table. Attempting to insert *H*. Linear probing suggests location 13, but that is too far, so we evict *G* from position 11 to find a closer position.

	Item	Hop
...		
6	C	1000
7	A	1100
8	D	0010
9	B	1010
10	E	0000
11	H	0010
12	F	1000
13	G	0000
14		0000
...		

→

	Item	Hop
...		
6	C	1000
7	A	1100
8	D	0010
9	B	1010
10	E	0000
11	H	**0001**
12	F	1000
13		0000
14	**G**	0000
...		

→

	Item	Hop
...		
6	C	1000
7	A	1100
8	D	0010
9	B	1010
10	E	0000
11	H	0001
12		**0100**
13	**F**	0000
14	G	0000
...		

A: 7
B: 9
C: 6
D: 7
E: 8
F: 12
G: 11
H: 9
I: 6

Figure 5.48 Hopscotch hashing table. Attempting to insert *I*. Linear probing suggests location 14, but that is too far; consulting Hop[11], we see that *G* can move down, leaving position 13 open. Consulting Hop[10] gives no suggestions. Hop[11] does not help either (why?), so Hop[12] suggests moving *F*.

Hopscotch hashing is a relatively new algorithm, but the initial experimental results are very promising, especially for applications that make use of multiple processors and require significant parallelism and concurrency. It remains to be seen if either cuckoo hashing or hopscotch hashing emerge as a practical alternative to the classic separate chaining and linear/quadratic probing schemes.

	Item	Hop
...		
6	C	1000
7	A	1100
8	D	0010
9	B	1010
10	E	0000
11	H	0001
12		0100
13	F	0000
14	G	0000
...		

→

	Item	Hop
...		
6	C	1000
7	A	1100
8	D	0010
9		**0011**
10	E	0000
11	H	0001
12	**B**	0100
13	F	0000
14	G	0000
...		

→

	Item	Hop
...		
6	C	**1001**
7	A	1100
8	D	0010
9	**I**	0011
10	E	0000
11	H	0001
12	B	0100
13	F	0000
14	G	0000
...		

A: 7
B: 9
C: 6
D: 7
E: 8
F: 12
G: 11
H: 9
I: 6

Figure 5.49 Hopscotch hashing table. Insertion of *I* continues: Next, *B* is evicted, and finally, we have a spot that is close enough to the hash value and can insert *I*.

5.8 Universal Hashing

Although hash tables are very efficient and have constant average cost per operation, assuming appropriate load factors, their analysis and performance depend on the hash function having two fundamental properties:

1. The hash function must be computable in constant time (i.e., independent of the number of items in the hash table).
2. The hash function must distribute its items uniformly among the array slots.

In particular, if the hash function is poor, then all bets are off, and the cost per operation can be linear. In this section, we discuss **universal hash functions**, which allow us to choose the hash function randomly in such a way that condition 2 above is satisfied. As in Section 5.7, we use M to represent *TableSize*. Although a strong motivation for the use of universal hash functions is to provide theoretical justification for the assumptions used in the classic hash table analyses, these functions can also be used in applications that require a high level of robustness, in which worst-case (or even substantially degraded) performance, perhaps based on inputs generated by a saboteur or hacker, simply cannot be tolerated.

As in Section 5.7, we use M to represent *TableSize*.

Definition 5.1

A family H of hash functions is *universal*, if for any $x \neq y$, the number of hash functions h in H for which $h(x) = h(y)$ is at most $|H|/M$.

Notice that this definition holds for each pair of items, rather than being averaged over all pairs of items. The definition above means that if we choose a hash function randomly from a universal family H, then the probability of a collision between any two distinct items

is at most $1/M$, and when adding into a table with N items, the probability of a collision at the initial point is at most N/M, or the load factor.

The use of a universal hash function for separate chaining or hopscotch hashing would be sufficient to meet the assumptions used in the analysis of those data structures. However, it is not sufficient for cuckoo hashing, which requires a stronger notion of independence. In cuckoo hashing, we first see if there is a vacant location; if there is not, and we do an eviction, a different item is now involved in looking for a vacant location. This repeats until we find the vacant location, or decide to rehash [generally within $O(\log N)$ steps]. In order for the analysis to work, each step must have a collision probability of N/M independently, with a different item x being subject to the hash function. We can formalize this independence requirement in the following definition.

Definition 5.2

A family H of hash functions is *k-universal*, if for any $x_1 \neq y_1, x_2 \neq y_2, \ldots, x_k \neq y_k$, the number of hash functions h in H for which $h(x_1) = h(y_1)$, $h(x_2) = h(y_2), \ldots$, and $h(x_k) = h(y_k)$ is at most $|H|/M^k$.

With this definition, we see that the analysis of cuckoo hashing requires an $O(\log N)$-universal hash function (after that many evictions, we give up and rehash). In this section we look only at universal hash functions.

To design a simple universal hash function, we will assume first that we are mapping very large integers into smaller integers ranging from 0 to $M - 1$. Let p be a prime larger than the largest input key.

Our universal family H will consist of the following set of functions, where a and b are chosen randomly:

$$H = \{H_{a,b}(x) = ((ax + b) \mod p) \mod M, \text{ where } 1 \leq a \leq p - 1, 0 \leq b \leq p - 1\}$$

For example, in this family, three of the possible random choices of (a, b) yield three different hash functions:

$$H_{3,7}(x) = ((3x + 7) \bmod p) \bmod M$$
$$H_{4,1}(x) = ((4x + 1) \bmod p) \bmod M$$
$$H_{8,0}(x) = ((8x) \bmod p) \bmod M$$

Observe that there are $p(p - 1)$ possible hash functions that can be chosen.

Theorem 5.4

The hash family $H = \{H_{a,b}(x) = ((ax + b) \bmod p) \bmod M$, where $1 \leq a \leq p - 1$, $0 \leq b \leq p - 1\}$ is universal.

Proof

Let x and y be distinct values, with $x > y$, such that $H_{a,b}(x) = H_{a,b}(y)$.

Clearly if $(ax + b) \bmod p$ is equal to $(ay + b) \bmod p$, then we will have a collision. However, this cannot happen: Subtracting equations yields $a(x - y) \equiv 0 \pmod p$, which would mean that p divides a or p divides $x - y$, since p is prime. But neither can happen, since both a and $x - y$ are between 1 and $p - 1$.

So let $r = (ax + b) \bmod p$ and let $s = (ay + b) \bmod p$, and by the above argument, $r \neq s$. Thus there are p possible values for r, and for each r, there are $p - 1$ possible

values for s, for a total of $p(p-1)$ possible (r, s) pairs. Notice that the number of (a, b) pairs and the number of (r, s) pairs is identical; thus each (r, s) pair will correspond to exactly one (a, b) pair if we can solve for (a, b) in terms of r and s. But that is easy: As before, subtracting equations yields $a(x-y) \equiv (r-s) \pmod{p}$, which means that by multiplying both sides by the unique multiplicative inverse of $(x-y)$ (which must exist, since $x-y$ is not zero and p is prime), we obtain a, in terms of r and s. Then b follows.

Finally, this means that the probability that x and y collide is equal to the probability that $r \equiv s \pmod{M}$, and the above analysis allows us to assume that r and s are chosen randomly, rather than a and b. Immediate intuition would place this probability at $1/M$, but that would only be true if p were an exact multiple of M, and all possible (r, s) pairs were equally likely. Since p is prime, and $r \neq s$, that is not exactly true, so a more careful analysis is needed.

For a given r, the number of values of s that can collide mod M is at most $\lceil p/M \rceil - 1$ (the -1 is because $r \neq s$). It is easy to see that this is at most $(p-1)/M$. Thus the probability that r and s will generate a collision is at most $1/M$ (we divide by $p-1$, because, as mentioned earlier in the proof, there are only $p-1$ choices for s given r). This implies that the hash family is universal.

Implementation of this hash function would seem to require two mod operations: one mod p and the second mod M. Figure 5.50 shows a simple implementation in C++, assuming that M is significantly less than $2^{31}-1$. Because the computations must now be exactly as specified, and thus overflow is no longer acceptable, we promote to `long long` computations, which are at least 64 bits.

However, we are allowed to choose any prime p, as long as it is larger than M. Hence, it makes sense to choose a prime that is most favorable for computations. One such prime is $p = 2^{31}-1$. Prime numbers of this form are known as Mersenne primes; other Mersenne primes include 2^5-1, $2^{61}-1$ and $2^{89}-1$. Just as a multiplication by a Mersenne prime such as 31 can be implemented by a bit shift and a subtract, a mod operation involving a Mersenne prime can also be implemented by a bit shift and an addition:

Suppose $r \equiv y \pmod{p}$. If we divide y by $(p+1)$, then $y = q'(p+1) + r'$, where q' and r' are the quotient and remainder, respectively. Thus, $r \equiv q'(p+1) + r' \pmod{p}$. And since $(p+1) \equiv 1 \pmod{p}$, we obtain $r \equiv q' + r' \pmod{p}$.

Figure 5.51 implements this idea, which is known as the **Carter-Wegman trick**. On line 8, the bit shift computes the quotient and the bitwise-and computes the remainder when dividing by $(p+1)$; these bitwise operations work because $(p+1)$ is an exact power

```
int universalHash( int x, int A, int B, int P, int M )
{
    return static_cast<int>( ( ( static_cast<long long>( A ) * x ) + B ) % P ) % M;
}
```

Figure 5.50 Simple implementation of universal hashing

```
1  const int DIGS = 31;
2  const int mersennep = (1<<DIGS) - 1;
3
4  int universalHash( int x, int A, int B, int M )
5  {
6      long long hashVal = static_cast<long long>( A ) * x + B;
7
8      hashVal = ( ( hashVal >> DIGS ) + ( hashVal & mersennep ) );
9      if( hashVal >= mersennep )
10         hashVal -= mersennep;
11
12     return static_cast<int>( hashVal ) % M;
13 }
```

Figure 5.51 Simple implementation of universal hashing

of two. Since the remainder could be almost as large as p, the resulting sum might be larger than p, so we scale it back down at lines 9 and 10.

Universal hash functions exist for strings also. First, choose any prime p, larger than M (and larger than the largest character code). Then use our standard string hashing function, choosing the multiplier randomly between 1 and $p - 1$ and returning an intermediate hash value between 0 and $p - 1$, inclusive. Finally, apply a universal hash function to generate the final hash value between 0 and $M - 1$.

5.9 Extendible Hashing

Our last topic in this chapter deals with the case where the amount of data is too large to fit in main memory. As we saw in Chapter 4, the main consideration then is the number of disk accesses required to retrieve data.

As before, we assume that at any point we have N records to store; the value of N changes over time. Furthermore, at most M records fit in one disk block. We will use $M = 4$ in this section.

If either probing hashing or separate chaining hashing is used, the major problem is that collisions could cause several blocks to be examined during a search, even for a well-distributed hash table. Furthermore, when the table gets too full, an extremely expensive rehashing step must be performed, which requires $O(N)$ disk accesses.

A clever alternative, known as **extendible hashing**, allows a search to be performed in two disk accesses. Insertions also require few disk accesses.

We recall from Chapter 4 that a B-tree has depth $O(\log_{M/2} N)$. As M increases, the depth of a B-tree decreases. We could in theory choose M to be so large that the depth of the B-tree would be 1. Then any search after the first would take one disk access, since, presumably, the root node could be stored in main memory. The problem with this strategy is that the branching factor is so high that it would take considerable processing to determine which leaf the data was in. If the time to perform this step could be reduced, then we would have a practical scheme. This is exactly the strategy used by extendible hashing.

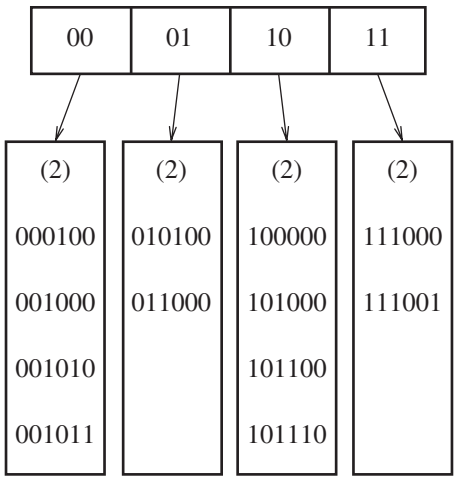

Figure 5.52 Extendible hashing: original data

Let us suppose, for the moment, that our data consists of several 6-bit integers. Figure 5.52 shows an extendible hashing scheme for these data. The root of the "tree" contains four pointers determined by the leading two bits of the data. Each leaf has up to $M = 4$ elements. It happens that in each leaf the first two bits are identical; this is indicated by the number in parentheses. To be more formal, D will represent the number of bits used by the root, which is sometimes known as the **directory.** The number of entries in the directory is thus 2^D. d_L is the number of leading bits that all the elements of some leaf L have in common. d_L will depend on the particular leaf, and $d_L \leq D$.

Suppose that we want to insert the key 100100. This would go into the third leaf, but as the third leaf is already full, there is no room. We thus split this leaf into two leaves, which are now determined by the first *three* bits. This requires increasing the directory size to 3. These changes are reflected in Figure 5.53.

Notice that all the leaves not involved in the split are now pointed to by two adjacent directory entries. Thus, although an entire directory is rewritten, none of the other leaves is actually accessed.

If the key 000000 is now inserted, then the first leaf is split, generating two leaves with $d_L = 3$. Since $D = 3$, the only change required in the directory is the updating of the 000 and 001 pointers. See Figure 5.54.

This very simple strategy provides quick access times for `insert` and search operations on large databases. There are a few important details we have not considered.

First, it is possible that several directory splits will be required if the elements in a leaf agree in more than $D + 1$ leading bits. For instance, starting at the original example, with $D = 2$, if 111010, 111011, and finally 111100 are inserted, the directory size must be increased to 4 to distinguish between the five keys. This is an easy detail to take care of, but must not be forgotten. Second, there is the possibility of duplicate keys; if there are more than M duplicates, then this algorithm does not work at all. In this case, some other arrangements need to be made.

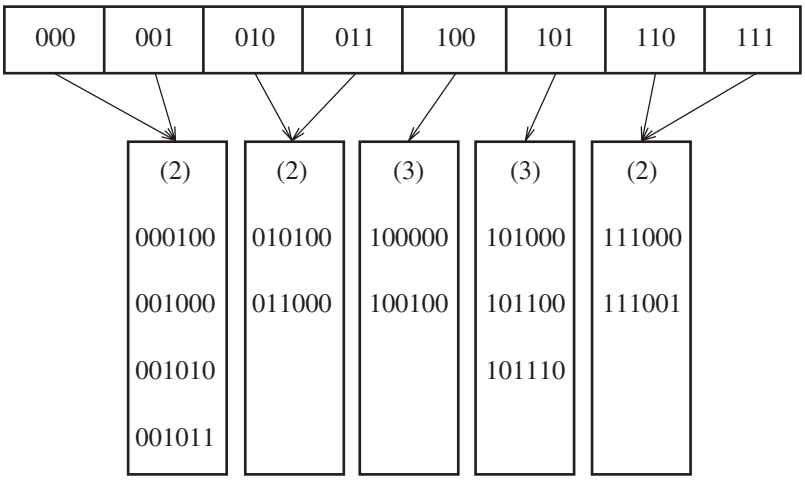

Figure 5.53 Extendible hashing: after insertion of 100100 and directory split

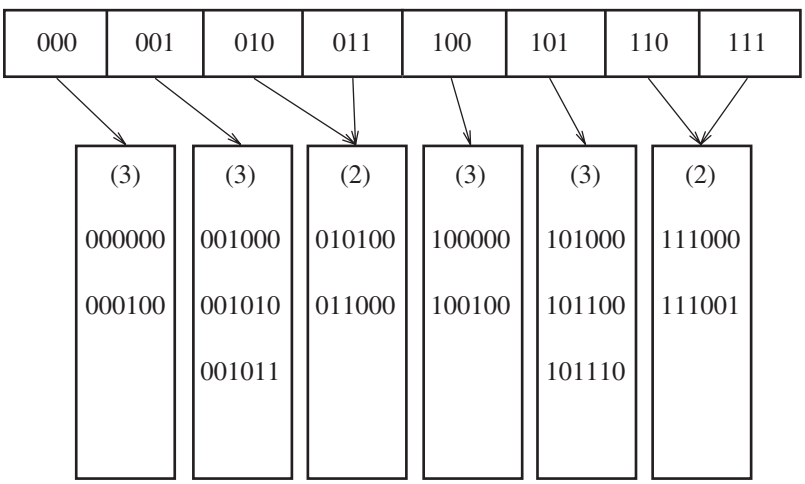

Figure 5.54 Extendible hashing: after insertion of 000000 and leaf split

These possibilities suggest that it is important for the bits to be fairly random. This can be accomplished by hashing the keys into a reasonably long integer—hence the name.

We close by mentioning some of the performance properties of extendible hashing, which are derived after a very difficult analysis. These results are based on the reasonable assumption that the bit patterns are uniformly distributed.

The expected number of leaves is $(N/M)\log_2 e$. Thus the average leaf is $\ln 2 = 0.69$ full. This is the same as for B-trees, which is not entirely surprising, since for both data structures new nodes are created when the $(M + 1)$th entry is added.

The more surprising result is that the expected size of the directory (in other words, 2^D) is $O(N^{1+1/M}/M)$. If M is very small, then the directory can get unduly large. In this case, we can have the leaves contain pointers to the records instead of the actual records, thus increasing the value of M. This adds a second disk access to each search operation in order to maintain a smaller directory. If the directory is too large to fit in main memory, the second disk access would be needed anyway.

Summary

Hash tables can be used to implement the `insert` and `contains` operations in constant average time. It is especially important to pay attention to details such as load factor when using hash tables, since otherwise the time bounds are not valid. It is also important to choose the hash function carefully when the key is not a short string or integer.

For separate chaining hashing, the load factor should be close to 1, although performance does not significantly degrade unless the load factor becomes very large. For probing hashing, the load factor should not exceed 0.5, unless this is completely unavoidable. If linear probing is used, performance degenerates rapidly as the load factor approaches 1. Rehashing can be implemented to allow the table to grow (and shrink), thus maintaining a reasonable load factor. This is important if space is tight and it is not possible just to declare a huge hash table.

Other alternatives such as cuckoo hashing and hopscotch hashing can also yield good results. Because all these algorithms are constant time, it is difficult to make strong statements about which hash table implementation is the "best"; recent simulation results provide conflicting guidance and suggest that the performance can depend strongly on the types of items being manipulated, the underlying computer hardware, and the programming language.

Binary search trees can also be used to implement `insert` and `contains` operations. Although the resulting average time bounds are $O(\log N)$, binary search trees also support routines that require order and are thus more powerful. Using a hash table, it is not possible to find the minimum element. It is not possible to search efficiently for a string unless the exact string is known. A binary search tree could quickly find all items in a certain range; this is not supported by hash tables. Furthermore, the $O(\log N)$ bound is not necessarily that much more than $O(1)$, especially since no multiplications or divisions are required by search trees.

On the other hand, the worst case for hashing generally results from an implementation error, whereas sorted input can make binary trees perform poorly. Balanced search trees are quite expensive to implement, so if no ordering information is required and there is any suspicion that the input might be sorted, then hashing is the data structure of choice.

Hashing applications are abundant. Compilers use hash tables to keep track of declared variables in source code. The data structure is known as a **symbol table.** Hash tables are the ideal application for this problem. Identifiers are typically short, so the hash function can be computed quickly, and alphabetizing the variables is often unnecessary.

A hash table is useful for any graph theory problem where the nodes have real names instead of numbers. Here, as the input is read, vertices are assigned integers from 1 onward by order of appearance. Again, the input is likely to have large groups of alphabetized entries. For example, the vertices could be computers. Then if one particular installation lists its computers as *ibm1, ibm2, ibm3,* . . . , there could be a dramatic effect on efficiency if a search tree is used.

A third common use of hash tables is in programs that play games. As the program searches through different lines of play, it keeps track of positions it has seen by computing a hash function based on the position (and storing its move for that position). If the same position recurs, usually by a simple transposition of moves, the program can avoid expensive recomputation. This general feature of all game-playing programs is known as the **transposition table.**

Yet another use of hashing is in online spelling checkers. If misspelling detection (as opposed to correction) is important, an entire dictionary can be prehashed and words can be checked in constant time. Hash tables are well suited for this, because it is not important to alphabetize words; printing out misspellings in the order they occurred in the document is certainly acceptable.

Hash tables are often used to implement caches, both in software (for instance, the cache in your Internet browser) and in hardware (for instance, the memory caches in modern computers). They are also used in hardware implementations of routers.

We close this chapter by returning to the word puzzle problem of Chapter 1. If the second algorithm described in Chapter 1 is used, and we assume that the maximum word size is some small constant, then the time to read in the dictionary containing W words and put it in a hash table is $O(W)$. This time is likely to be dominated by the disk I/O and not the hashing routines. The rest of the algorithm would test for the presence of a word for each ordered quadruple (*row, column, orientation, number of characters*). As each lookup would be $O(1)$, and there are only a constant number of orientations (8) and characters per word, the running time of this phase would be $O(R \cdot C)$. The total running time would be $O(R \cdot C + W)$, which is a distinct improvement over the original $O(R \cdot C \cdot W)$. We could make further optimizations, which would decrease the running time in practice; these are described in the exercises.

Exercises

5.1 Given input {4371, 1323, 6173, 4199, 4344, 9679, 1989} and a hash function $h(x) = x$ (mod () 10), show the resulting
a. separate chaining hash table
b. hash table using linear probing
c. hash table using quadratic probing
d. hash table with second hash function $h_2(x) = 7 - (x \mod 7)$

5.2 Show the result of rehashing the hash tables in Exercise 5.1.

5.3 Write a program to compute the number of collisions required in a long random sequence of insertions using linear probing, quadratic probing, and double hashing.

5.4 A large number of deletions in a separate chaining hash table can cause the table to be fairly empty, which wastes space. In this case, we can rehash to a table half as large. Assume that we rehash to a larger table when there are twice as many elements as the table size. How empty should the table be before we rehash to a smaller table?

5.5 Reimplement separate chaining hash tables using a `vector` of singly linked lists instead of `vector`s.

5.6 The `isEmpty` routine for quadratic probing has not been written. Can you implement it by returning the expression `currentSize==0`?

5.7 In the quadratic probing hash table, suppose that instead of inserting a new item into the location suggested by `findPos`, we insert it into the first inactive cell on the search path (thus, it is possible to reclaim a cell that is marked deleted, potentially saving space).

a. Rewrite the insertion algorithm to use this observation. Do this by having `findPos` maintain, with an additional variable, the location of the first inactive cell it encounters.

b. Explain the circumstances under which the revised algorithm is faster than the original algorithm. Can it be slower?

5.8 Suppose instead of quadratic probing, we use "cubic probing"; here the ith probe is at $hash(x) + i^3$. Does cubic probing improve on quadratic probing?

5.9 Using a standard dictionary, and a table size that approximates a load factor of 1, compare the number of collisions produced by the hash function in Figure 5.4 and the hash function in Figure 5.55.

5.10 What are the advantages and disadvantages of the various collision resolution strategies?

5.11 Suppose that to mitigate the effects of secondary clustering we use as the collision resolution function $f(i) = i \cdot r(hash(x))$, where $hash(x)$ is the 32-bit hash value (not yet scaled to a suitable array index), and $r(y) = |48271y(\mod (2^{31} - 1))|$

```
/**
 * FNV-1a hash routine for string objects.
 */
unsigned int hash( const string & key, int tableSize )
{
    unsigned int hashVal = 2166136261;

    for( char ch : key )
        hashVal = ( hashVal ^ ch )* 16777619;

    return hashVal % tableSize;
}
```

Figure 5.55 Alternative hash function for Exercise 5.9.

mod *TableSize*. (Section 10.4.1 describes a method of performing this calculation without overflows, but it is unlikely that overflow matters in this case.) Explain why this strategy tends to avoid secondary clustering, and compare this strategy with both double hashing and quadratic probing.

5.12 Rehashing requires recomputing the hash function for all items in the hash table. Since computing the hash function is expensive, suppose objects provide a hash member function of their own, and each object stores the result in an additional data member the first time the hash function is computed for it. Show how such a scheme would apply for the `Employee` class in Figure 5.8, and explain under what circumstances the remembered hash value remains valid in each `Employee`.

5.13 Write a program to implement the following strategy for multiplying two sparse polynomials P_1, P_2 of size M and N, respectively. Each polynomial is represented as a list of objects consisting of a coefficient and an exponent. We multiply each term in P_1 by a term in P_2 for a total of MN operations. One method is to sort these terms and combine like terms, but this requires sorting MN records, which could be expensive, especially in small-memory environments. Alternatively, we could merge terms as they are computed and then sort the result.

a. Write a program to implement the alternative strategy.
b. If the output polynomial has about $O(M + N)$ terms, what is the running time of both methods?

* **5.14** Describe a procedure that avoids initializing a hash table (at the expense of memory).

5.15 Suppose we want to find the first occurrence of a string $P_1P_2 \cdots P_k$ in a long input string $A_1A_2 \cdots A_N$. We can solve this problem by hashing the pattern string, obtaining a hash value H_P, and comparing this value with the hash value formed from $A_1A_2 \cdots A_k, A_2A_3 \cdots A_{k+1}, A_3A_4 \cdots A_{k+2}$, and so on until $A_{N-k+1}A_{N-k+2} \cdots A_N$. If we have a match of hash values, we compare the strings character by character to verify the match. We return the position (in A) if the strings actually do match, and we continue in the unlikely event that the match is false.

* a. Show that if the hash value of $A_iA_{i+1} \cdots A_{i+k-1}$ is known, then the hash value of $A_{i+1}A_{i+2} \cdots A_{i+k}$ can be computed in constant time.
b. Show that the running time is $O(k + N)$ plus the time spent refuting false matches.
* c. Show that the expected number of false matches is negligible.
d. Write a program to implement this algorithm.
* e. Describe an algorithm that runs in $O(k + N)$ worst-case time.
** f. Describe an algorithm that runs in $O(N/k)$ average time.

5.16 A nonstandard C++ extension adds syntax that allows a switch statement to work with the `string` type (instead of the primitive integer types). Explain how hash tables can be used by the compiler to implement this language addition.

5.17 An (old-style) BASIC program consists of a series of statements numbered in ascending order. Control is passed by use of a *goto* or *gosub* and a statement number. Write a program that reads in a legal BASIC program and renumbers the statements so

that the first starts at number F and each statement has a number D higher than the previous statement. You may assume an upper limit of N statements, but the statement numbers in the input might be as large as a 32-bit integer. Your program must run in linear time.

5.18 a. Implement the word puzzle program using the algorithm described at the end of the chapter.

b. We can get a big speed increase by storing, in addition to each word W, all of W's prefixes. (If one of W's prefixes is another word in the dictionary, it is stored as a real word.) Although this may seem to increase the size of the hash table drastically, it does not, because many words have the same prefixes. When a scan is performed in a particular direction, if the word that is looked up is not even in the hash table as a prefix, then the scan in that direction can be terminated early. Use this idea to write an improved program to solve the word puzzle.

c. If we are willing to sacrifice the sanctity of the hash table ADT, we can speed up the program in part (b) by noting that if, for example, we have just computed the hash function for "excel," we do not need to compute the hash function for "excels" from scratch. Adjust your hash function so that it can take advantage of its previous calculation.

d. In Chapter 2, we suggested using binary search. Incorporate the idea of using prefixes into your binary search algorithm. The modification should be simple. Which algorithm is faster?

5.19 Under certain assumptions, the expected cost of an insertion into a hash table with secondary clustering is given by $1/(1-\lambda)-\lambda-\ln(1-\lambda)$. Unfortunately, this formula is not accurate for quadratic probing. However, assuming that it is, determine the following:

a. the expected cost of an unsuccessful search

b. the expected cost of a successful search

5.20 Implement a generic `Map` that supports the `insert` and `lookup` operations. The implementation will store a hash table of pairs (key, definition). You will `lookup` a definition by providing a key. Figure 5.56 provides the `Map` specification (minus some details).

5.21 Implement a spelling checker by using a hash table. Assume that the dictionary comes from two sources: an existing large dictionary and a second file containing a personal dictionary. Output all misspelled words and the line numbers on which they occur. Also, for each misspelled word, list any words in the dictionary that are obtainable by applying any of the following rules:

a. Add one character.

b. Remove one character.

c. Exchange adjacent characters.

5.22 Prove **Markov's Inequality**: If X is any random variable and $a > 0$, then $\Pr(|X| \geq a) \leq E(|X|)/a$. Show how this inequality can be applied to Theorems 5.2 and 5.3.

5.23 If a hopscotch table with parameter *MAX_DIST* has load factor 0.5, what is the approximate probability that an insertion requires a rehash?